Magic Methods

Module 1: Python Foundations

```
class BoardGame:
    def __str__(self):
        return "Uno, 8 left"
print(BoardGame())
```

By the End of Today, You'll Be Able To…

1

Add `__str__` and `__repr__` to a class

2

Explain which one Python actually uses

You'll Know These by the End of Today

Magic method

special Python method

`__str__`

friendly display

`__repr__`

precise representation

Reading Recap: Two Ways to Introduce an Object

Today's Reading: Magic Methods

A pet-sitting app prints a Pet object and gets a memory address, not the pet. Reopen the reading: Two Ways to Introduce an Object

The Scenario: A Pet-Sitting App

No methods: `print(biscuit)` shows a memory address. Which pet is it?

For owners, `__str__` gives: Biscuit the dog

For programmers, `__repr__` gives: `Pet('Biscuit', 'dog')`

READING RECAP

Key Terms and Code From the Reading

MAGIC METHOD
Starts and ends with two underscores, like `__init__`. Python calls it for you.

`__str__`
A friendly string for people. `print(obj)` uses it.

`__repr__`
An exact string for programmers. `repr(obj)` and lists use it.

FALLBACK RULE
No `__str__`? Then `print()` uses `__repr__`.

READING CHECK

A Pet class has `__repr__` but no `__str__`. What does `print(biscuit)` show?

A

An error, because `__str__` is missing

B

`Pet('Biscuit', 'dog')`

C

Class name plus memory address

ANSWER KEY

Answer Key: The Fallback Rule

Correct answer: B. `Pet('Biscuit', 'dog')`

`print()` asks for `__str__` first. With none defined, Python falls back to `__repr__`. No error.

READING CHECK

`Visit("Biscuit", 30)` made the object. Which `repr()` output follows the reading's style?

A

```
Visit(Biscuit, 30)
```

B

```
Visit('Biscuit', '30')
```

C

```
Visit('Biscuit', 30)
```

ANSWER KEY

Answer Key: Quotes on Text, Not on Numbers

Correct answer: C. `Visit('Biscuit', 30)`

The usual `__repr__` style looks like the code that made the object: quotes around the text value, none around the number.

Your Questions From the Reading

Turn and talk for 1 min. Bring one question.

We take 2 or 3 questions now.

Bigger ones wait for today's matching slide.

YESTERDAY'S CLASS

Partner Up Again

Same partner, same `BoardGame` class — today adds two more methods on top of it.

QUICK RECALL

Is each piece a module, class, object, method or attribute?

A

`connect4`

B

`__str__`

C

`board_games.py`

D

`self.title`

E

`BoardGame`

F

`__init__`

ANSWER KEY

Each Piece, Labeled

A

`connect4`
object

B

`__str__`
method

C

`board_games.py`
module

D

`self.title`
attribute

E

`BoardGame`
class

F

`__init__`
method

Printing an Object Right Now Is Useless

DEFAULT

`print(connect4)` shows almost nothing useful.

Printing an Object Right Now Is Useless

DEFAULT

`print(connect4)` shows almost nothing useful.

TODAY

Replace the memory-address-like output with real object data.

Friendly Name vs. Exact ID

`__str__`

Like a nickname:
quick, friendly, recognizable.

`__repr__`

Like the exact name on an ID:
precise and unambiguous.

CHECK FOR UNDERSTANDING

In the ID analogy, which one is more like `__repr__`?

A The nickname

B Neither — they're the same

C The exact name on your ID

ANSWER KEY

ID Analogy Recall

Correct: The exact name on your ID

`__repr__` is the precise, unambiguous version — like an exact name that identifies you without confusion.

Which One Does Python Actually Use?

`print()` / `str()`

Reach for `__str__` first.

Which One Does Python Actually Use?

`print()` / `str()`

Reach for `__str__` first.

NO `__str__`?

Falls back to `__repr__`.

Which One Does Python Actually Use?

`print()` / `str()`

Reach for `__str__` first.

NO `__str__`?

Falls back to `__repr__`.

INSIDE A LIST

Python uses `__repr__`.

IN THE NEWS

His Plate Said **NULL**. $12,000 in Tickets Followed.

```
print("None")   # None
print(None)     # None
```

Same idea in Python: text or nothing? `print()` shows both as None.

Illustration, not the real ticket system

Tartaro picked NULL for his car plate
Tickets with no plate also read NULL
The system sent those tickets to him

On screen, the text NULL and no plate look the same. What would show the difference?

[Source: The Drive, on Joseph Tartaro's DEF CON 27 talk (August 2019)]

CODE ALONG

Open the Code-Along

Open this file:
`Module1_Lesson14_magic-methods-what-init-str-repr-actually-mean_STUDENT.py`
Run it after every section: **`python3`** + the file name

Sections, in order:

1. Watch This: The Default Is Ugly (watch-only)
2. Step 1: Add **`__str__`** · one method fixes what `print(connect4)` shows
3. Step 2: Add **`__repr__`** Too · now add the precise version too
4. Now Test **`repr()`** (watch-only)
5. The Real Gotcha: A List of Objects · alone vs. inside a list: which method runs?

Watch This: The Default Is Ugly

python

```python
class BoardGame:
    def __init__(self, title, players_available,
                 category):
        self.title = title
        self.players_available = players_available
        self.category = category

connect4 = BoardGame("Connect Four", 4, "Strategy")
print(connect4)
```

Step 1: Add `__str__`

python

```python
class BoardGame:
    def __init__(self, title, players_available,
                 category):
        self.title = title
        self.players_available = players_available
        self.category = category
    def __str__(self):
        return f"{self.title}, {self.players_available} left"
connect4 = BoardGame("Connect Four", 4, "Strategy")
print(connect4)
```

Step 2: Add `__repr__` Too

python

```python
class BoardGame:
    def __init__(self, title, players_available,
                 category):
        self.title = title
        self.players_available = players_available
        self.category = category
    def __str__(self):
        return f"{self.title}, {self.players_available} left"
    def __repr__(self):
        return f"BoardGame('{self.title}')"
```

Now Test `repr()`

```python
connect4 = BoardGame("Connect Four", 4, "Strategy")
print(repr(connect4))
```

The Real Gotcha: A List of Objects

```python
connect4 = BoardGame("Connect Four", 4, "Strategy")
uno = BoardGame("Uno", 8, "Party")

print(connect4)
print([connect4, uno])
```

CODE-ALONG DONE

Back to the Slides: Review

You just typed it and ran it. Now check that it stuck, without the code in front of you.

QUICK CHALLENGE

The Step 1 code never calls `__str__`. Why does `print(connect4)` change?

A `print()` calls `__str__` automatically

B `__init__` calls `__str__` for you

C The f-string changes `print()`

ANSWER KEY

Who Calls `__str__`?

Correct: `print()` calls `__str__` automatically

You never type `connect4.__str__()`. `print()` calls `__str__` for you whenever the class defines one. Before Step 1, nothing was defined, so `print()` fell back to the default.

Watch It Run: `print()` Calls `__str__`

Step 1 of 5

connect4 is built: title "Connect Four", 4 left

```
class BoardGame:  # __init__ as in Step 1
    def __str__(self):
        return f"{self.title}, {self.players_available} left"
connect4 = BoardGame("Connect Four", 4, "Strategy")
print(connect4)
```

OUTPUT

ANIMATED IN CLASS: watch it run in the lecture recording

QUICK CHALLENGE

Why did the Now Test repr() code print `BoardGame('Connect Four')`?

A `repr()` calls `__repr__`, not `__str__`

B `__repr__` replaced `__str__`

C `print()` skips `__str__` once `__repr__` exists

ANSWER KEY

Why `repr()` Skips `__str__`

Correct: `repr()` calls `__repr__`, not `__str__`

Both methods live on the class. `repr()` asks for `__repr__` specifically, so `__str__` never runs. `print(connect4)` alone still shows `"Connect Four, 4 left"`.

QUICK CHALLENGE

Inside `print([connect4, uno])`, which method runs for each object?

A `__str__`

B `__repr__`

C Neither — objects print raw

ANSWER KEY

The List Gotcha, Confirmed

Correct: `__repr__`

A list displays each item using its representation, which is why an object alone can look different from the same object inside a list.

Watch It Run: A List Uses `__repr__`

Step 1 of 5

This time print gets a list holding both games

```
class BoardGame:  # __init__ as in Step 1
    def __str__(self):
        return f"{self.title}, {self.players_available} left"
    def __repr__(self):
        return f"BoardGame('{self.title}')"
connect4 = BoardGame("Connect Four", 4, "Strategy")
uno = BoardGame("Uno", 8, "Party")
print([connect4, uno])
```

OUTPUT

ANIMATED IN CLASS: watch it run in the lecture recording

`__str__` vs. `__repr__`

`__str__`

- For a person reading it
- Used by `print()` and `str()`

`__repr__`

- For precision and debugging
- Used inside lists and as fallback

TIP

“Would You Define Only `__str__`, or Both?”

ONLY `__str__`

Friendly output for people.

BOTH

Friendly output + precise representation.

Answer like an interview, with the Tradeoff Lens: option, benefit, cost, when you'd choose it.

MODEL ANSWER

Model Answer: Tradeoff Lens

OPTION

Define both `__str__` and `__repr__` on the class.

BENEFIT

`print()` stays friendly; a list shows something precise.

COST

One more method to write and keep up to date.

WHEN I'D CHOOSE IT

Any class I'll debug or put in a list; skip only throwaways.

Key Terms — Recap

Magic method

special Python method

`__str__`

friendly display

`__repr__`

precise representation

So Far, We've Covered…

BEFORE

Built the class

TODAY

Added two more methods to that same class

NEXT

A solo design rep — no partner